

Name: Demarce Williams

Class: Signals and Systems

Lab #: 04

Date: September 18, 2025

Lab 4: Complex Poles

Solutions for Tasks 1–4

```
% Transfer function:
%  $H(z) = (1 - 0.9z^{-1} + 0.5z^{-2}) / (1 + 0.8z^{-2})$ 

clear; clc; close all;

% Coefficients of numerator and denominator
b = [1 -0.9 0.5]; % Numerator
a = [1 0 0.8];    % Denominator
```

Problem 1: Difference Equation & Impulse Response

```
fprintf('The difference equation for  $H(z) = Y(z)/X(z)$ \n');
```

The difference equation for $H(z) = Y(z)/X(z)$

```
fprintf('y(n) + 0.8*y(n-2) = x(n) - 0.9*x(n-1) + 0.5*x(n-2)\n');
```

$y(n) + 0.8*y(n-2) = x(n) - 0.9*x(n-1) + 0.5*x(n-2)$

```
fprintf('y(n) = x(n) - 0.9*x(n-1) + 0.5*x(n-2) - 0.8*y(n-2)\n');
```

$y(n) = x(n) - 0.9*x(n-1) + 0.5*x(n-2) - 0.8*y(n-2)$

```
fprintf('\n');
```

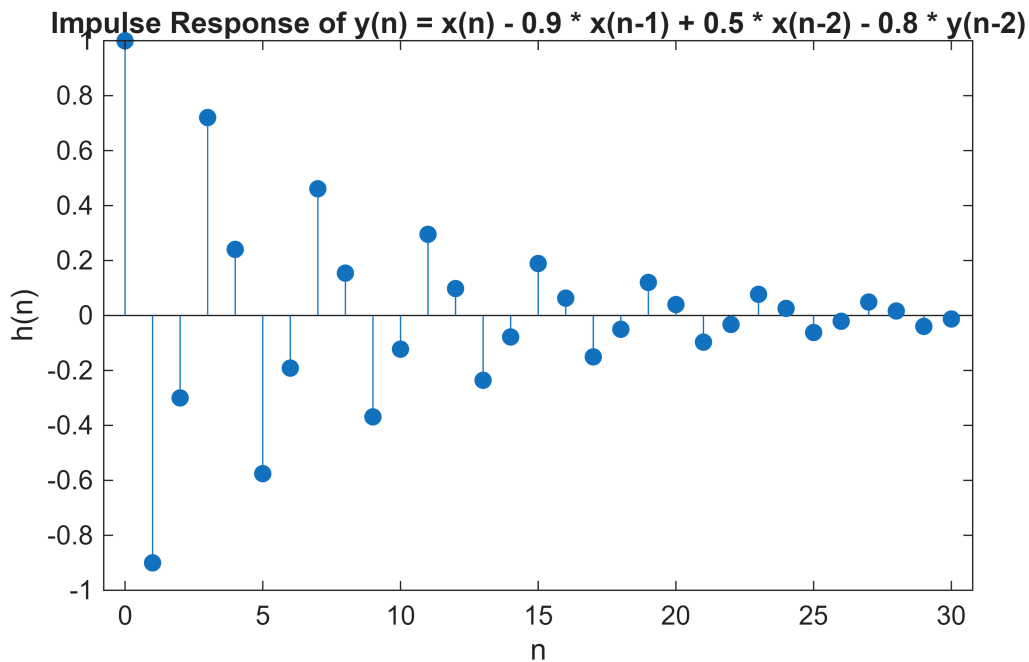
```
% Impulse input
N = 30; % number of samples
x = [1, zeros(1,N)];
y = mydiffreq(x);
```

```
% Plot impulse response
figure;
```

```

stem(0:N, y, 'filled');
title('Impulse Response of  $y(n) = x(n) - 0.9 * x(n-1) + 0.5 * x(n-2) - 0.8 * y(n-2)$ ');
xlabel('n');
ylabel('h(n)');

```

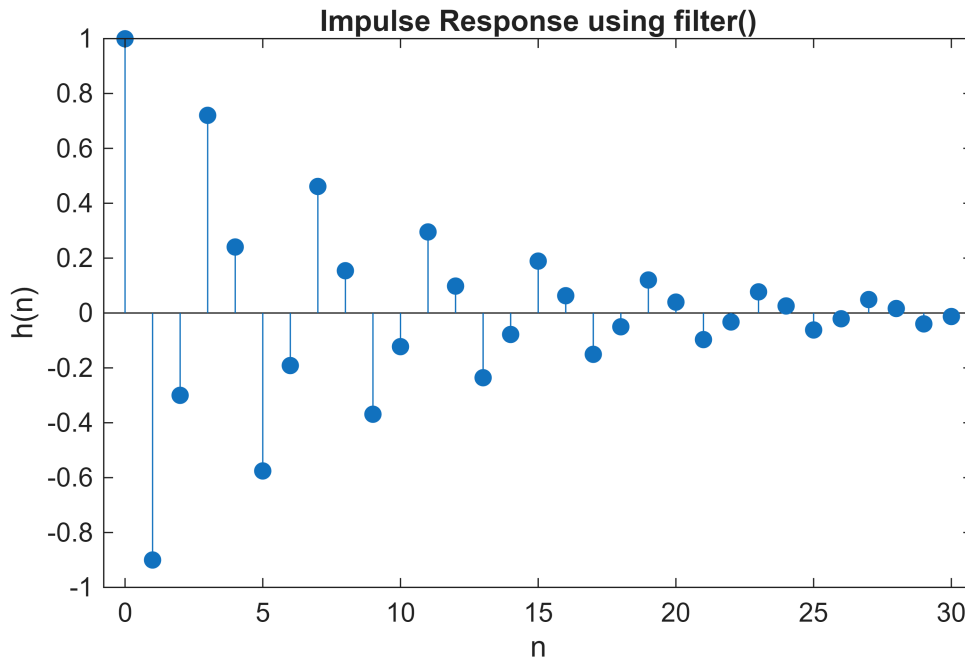


```

x = [1 zeros(1,N)]; % impulse input
y = filter(b, a, x); % using MATLAB built-in filter

figure;
stem(0:N, y, 'filled');
title('Impulse Response using filter()');
xlabel('n'); ylabel('h(n)');

```



```
disp('Both the difference Equation and the filter function gave the same impulse response.');
```

Both the difference Equation and the filter function gave the same impulse response.

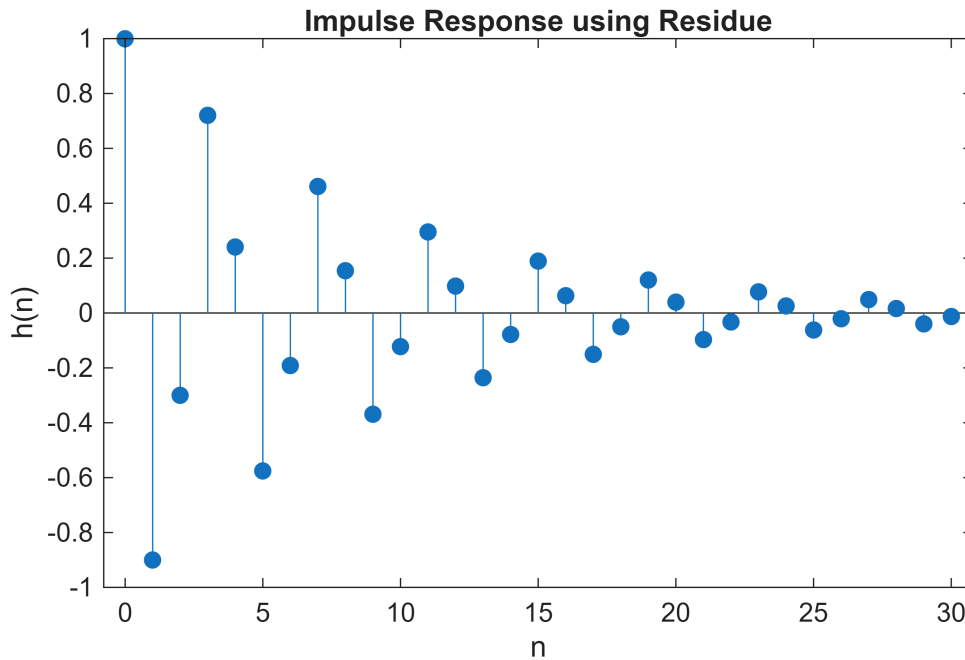
Problem 2: Partial Fraction Expansion (Residue)

```
% Adjust for H(z)/z: multiply denominator by [1 0] = z
a_Hz_over_z = conv(a, [1 0]);

% Do partial fraction expansion on H(z)/z
[C, p, k] = residue(b, a_Hz_over_z);

% Compute h(n)
n = -5:30;
h2 = zeros(size(n));
for i = 1:length(C)
    h2 = h2 + C(i) * (p(i).^n);
end

figure;
stem(n, h2, 'filled');
title('Impulse Response using Residue');
xlabel('n'); ylabel('h(n)');
```



```
disp('The residue function gave the same impulse response.');
```

The residue function gave the same impulse response.

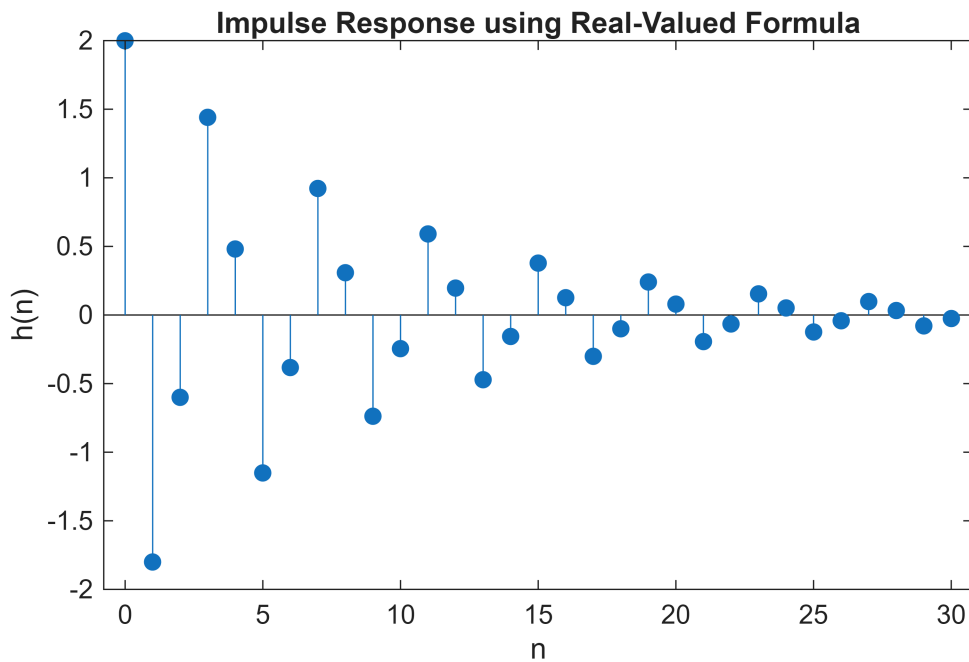
Problem 3: Real-Valued Formula (Damped Sinusoid)

```
% Convert residues and poles to polar form
R = abs(C);           % residue magnitudes
alpha = angle(C);     % residue angles
r = abs(p);           % pole magnitudes
phi = angle(p);       % pole angles

h3 = zeros(size(n)); % initialize

for i = 1:length(R)
    h3 = h3 + 2*R(i) * (r(i).^n) .* cos(phi(i)*n + alpha(i));
end

figure;
stem(n, h3, 'filled');
title('Impulse Response using Real-Valued Formula');
xlabel('n'); ylabel('h(n)');
```



Problem 4: Frequency of the Damped Cosine

```
% Damped cosine frequency in rad/sample
omega0 = phi(1); % frequency is the angle of pole p1

% Display results
fprintf('Pole p1 = %.4f + %.4fj\n', real(p(1)), imag(p(1)));
```

```
Pole p1 = -0.0000 + 0.8944j
```

```
fprintf('Magnitude (r1) = %.4f --> damping factor\n', r(1));
```

```
Magnitude (r1) = 0.8944 --> damping factor
```

```
fprintf('Angle (phi1) = %.4f rad/sample --> oscillation frequency\n', phi(1));
```

```
Angle (phi1) = 1.5708 rad/sample --> oscillation frequency
```

```
fprintf('Damped cosine frequency omega0 = %.4f rad/sample\n', omega0);
```

```
Damped cosine frequency omega0 = 1.5708 rad/sample
```